Project 1 - Verity

ML-P1-G15

Jan Francesc Porcar Guillen

Francesc Mateu Vallespín

Machine Learning – Year 25/26 – Verity – Version 1

Table of Contents

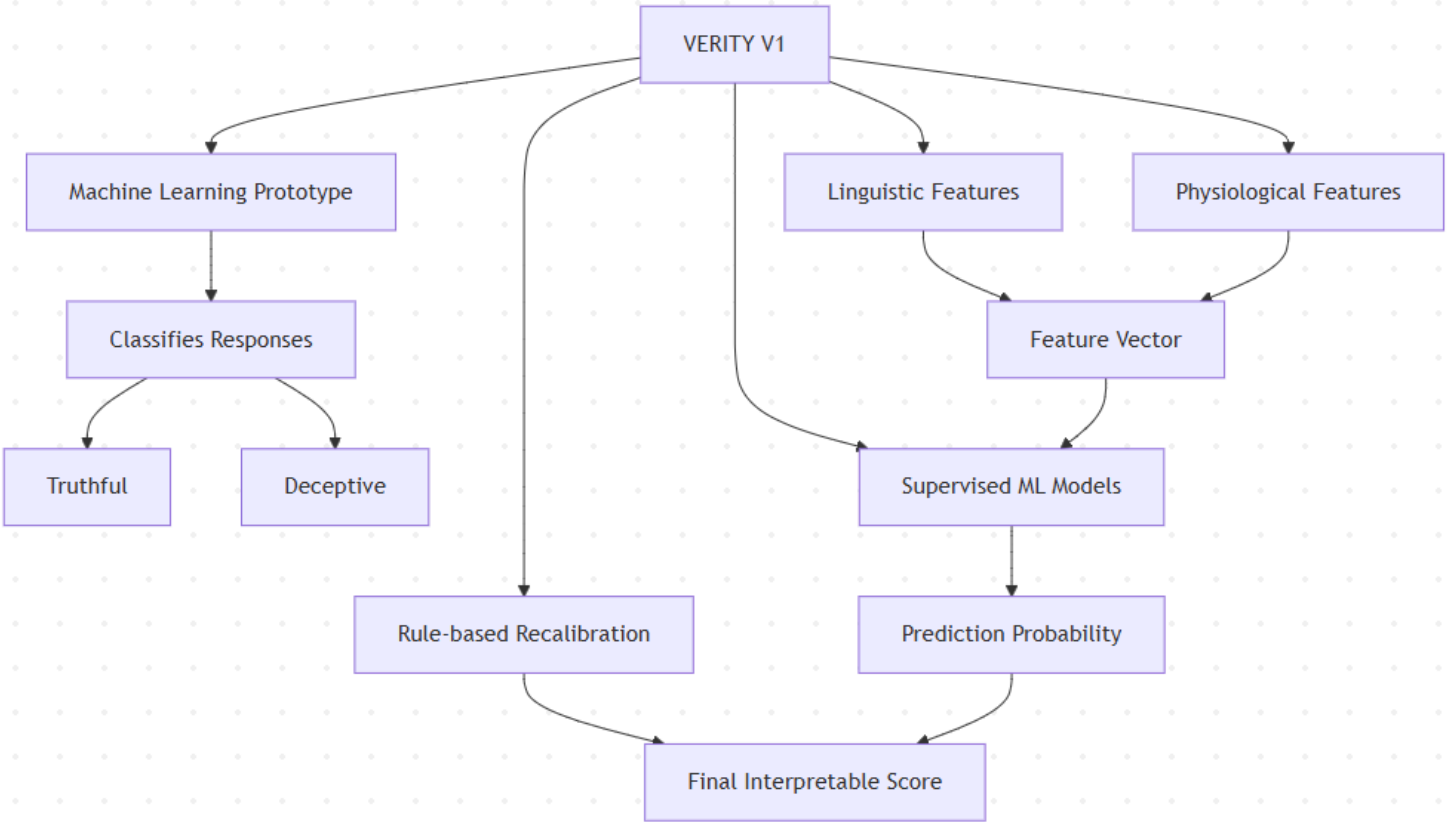
Introduction
General Pipeline
Dataset Design
Feature Engineering
Text Feature Extraction
Physiological Features
Data Preparation & Preprocessing
Machine Learning Algorithms
Model Evaluation & Interpretation
Recalibration Layer &Explainability
User Interface as Demonstration Layer
Limitations & Future Work
Conclusions

Introduction

VERITY V1 is a machine learning prototype that classifies whether users' answers are truth-based or deceitful. In the context of the project, the research aims to explore how various signals could be converted into numerical features that are further employed in supervised algorithms to predict the probability of deceitfulness. Contrary to most existing solutions, which use only written content generated by users, VERITY takes advantage of linguistic markers and fabricated physiological data.

The project cannot be considered a real lie detector. Indeed, its sole purpose is purely educational and experimental. First of all, it allows demonstrating how a comprehensive machine learning system could be developed step-by-step – including generating data, feature extraction, preparation, model development, algorithm comparison, evaluation, and linking the final outcome to an interactive application.

It is also vital to highlight that explainability is one of the crucial aspects of the project. The classifier does not merely produce outputs labeled 'truthful' or 'deceptive'. On the contrary, it deals with a wide range of features, including denial ratio, repetitions, specificity, heart rate, HRV, blink rate, and facial action units.

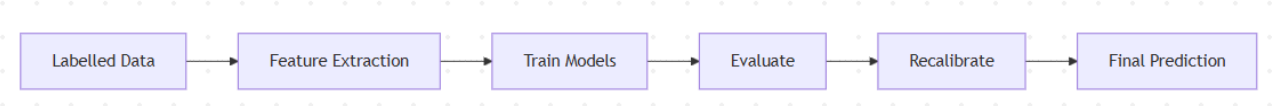


General Pipeline

The approach employed is based on supervised machine learning. The process begins with the presentation of a number of examples whose responses are labelled as truthful or deceptive. In this regard, the example responses are preprocessed such that the text information becomes a number of variables. Preprocessing is a vital aspect as some traditional machine learning techniques do not understand natural language text but rather operate through numeric variables.

Once the feature extraction is complete, a feature matrix X and a target vector y are generated. The feature matrix includes all the features used in the models while the target vector holds the correct classification for each of the samples. Next, the dataset is divided into a train and test sets, and the model learns from the training set.

In this regard, the final prediction is not merely the assignment of the class label. On the contrary, the models create probabilities which are followed by the application of a recalibration layer. This recalibration layer is a mixture of machine learning results and rule-based reasoning. Such a setup is important as it improves the interpretability and relevance of the result with respect to interface signals.



The key piece of code here is the distinction made between the input features and labels.

```
X = df[FEATURE_COLS].values
y = df["label"].values
```

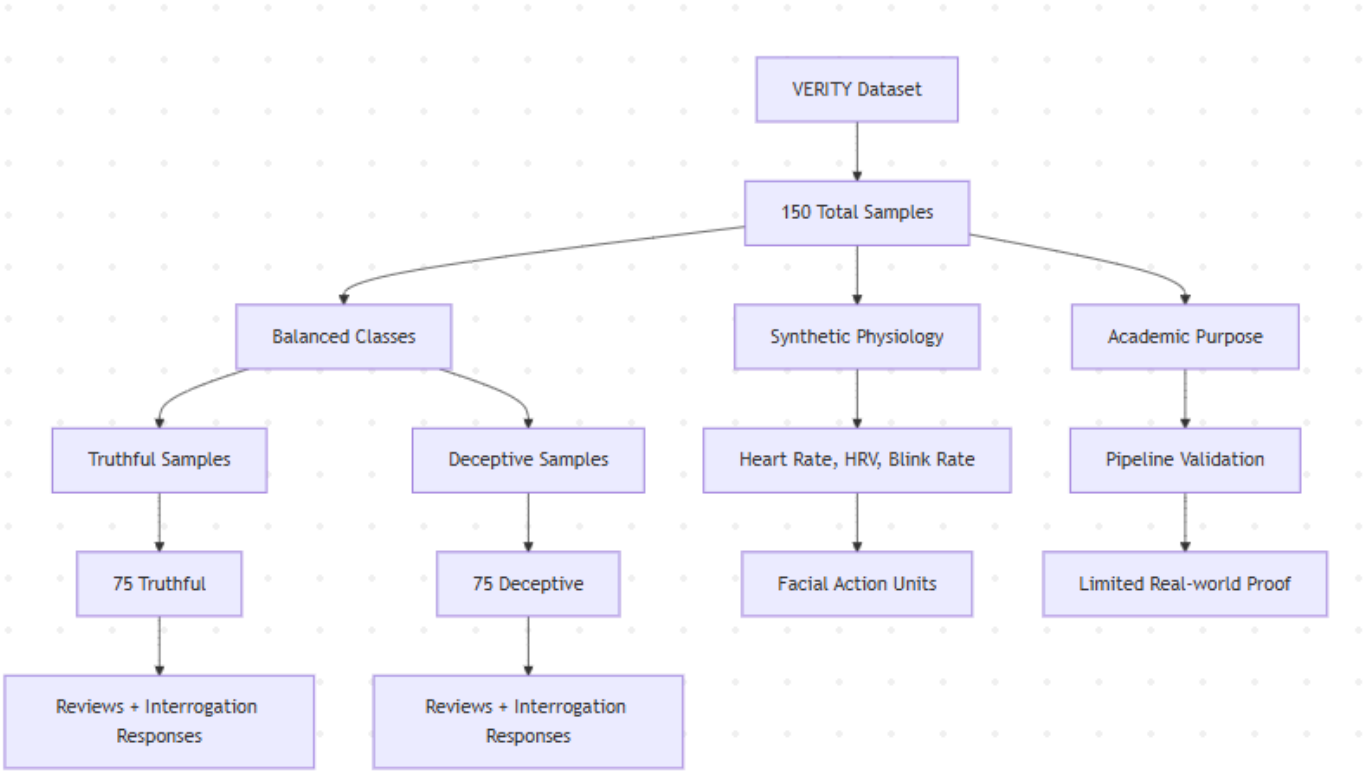
This illustrates how supervised learning works. The machine takes in as input the columns listed under FEATURE_COLS and tries to predict what is in the label.

Dataset Design

For VERITY to train effectively, the dataset used must have equal amounts of truths and lies, meaning 150 samples with 75 for each category. It is necessary because it ensures that the model does not learn any biased data. The bias could mean that if there are significantly more samples of one type, then the model will achieve high accuracy just by predicting the majority.

The combination of review and interrogation questions allows getting a wide range of language patterns for the model training. On the other hand, it poses an additional challenge because review texts and answers to the interrogation questions can hardly be similar to each other. Therefore, in order to get an effective algorithm for the detection of lies, the model should train on a larger, consistent, and homogeneous dataset.

It is also worth noting that, in contrast to physiological features from the real environment of deception, the values of heart rate, HRV, blink rate, and facial action units are simulated in the machine learning core. It has allowed completing the whole project, including the testing part of the pipeline, without using real cameras, sensors, and biometric devices.



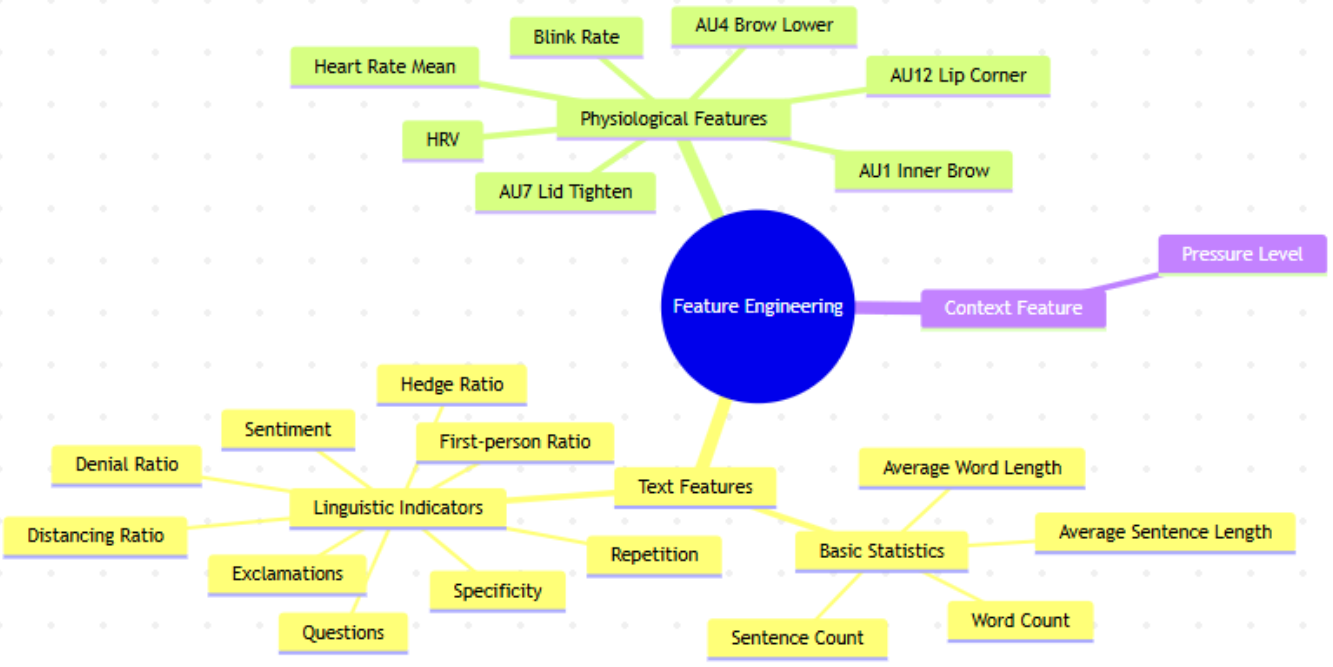
Feature Engineering

Probably the most critical element of the process is feature engineering. VERITY does not train the model based on the raw input – the response provided by the respondent. Rather, it extracts a set of numerical values that characterize the response from various angles. These variables will actually be used as the features for the models.

The text features include simple statistics (such as number of words, number of sentences, average word and average sentence lengths), but also more advanced linguistics features like the use of hedge words, denial words, distancing language, first-person ratio, questions, exclamations, repetition, specificity, and sentiment. All of these features are meant to capture linguistic tendencies associated with defensive language, vagueness, emotional exaggeration, uncertainty, or concrete details.

Finally, the physiological features include heart rate mean, HRV, blink rate, and several facial action units. These features aim to reflect stress, facial tension, or cognitive load. These are synthetic features in this iteration of the model, but their inclusion serves to illustrate a potential multimodal approach to the problem.

All these features are explicitly included in the code. This characteristic will come in handy during model interpretation as all input variables will be clearly identifiable in the source files.



Another highly important piece of code that needs to be included in the final report is the one describing the whole set of input variables used for building the models.

```
FEATURE_COLS = [
    "n_words", "n_sentences", "avg_word_len",
"avg_sentence_len",
    "hedge_count", "hedge_ratio", "denial_count",
"denial_ratio",
    "distancing_ratio", "first_person_ratio", "n_questions",
"n_exclamations",
    "repetition_ratio", "specificity_score",
"sentiment_ratio",
    "hr_mean", "hrv", "au1_inner_brow", "au4_brow_lower",
    "au7_lid_tighten", "au12_lip_corner", "blink_rate",
"pressure_level"
]
```

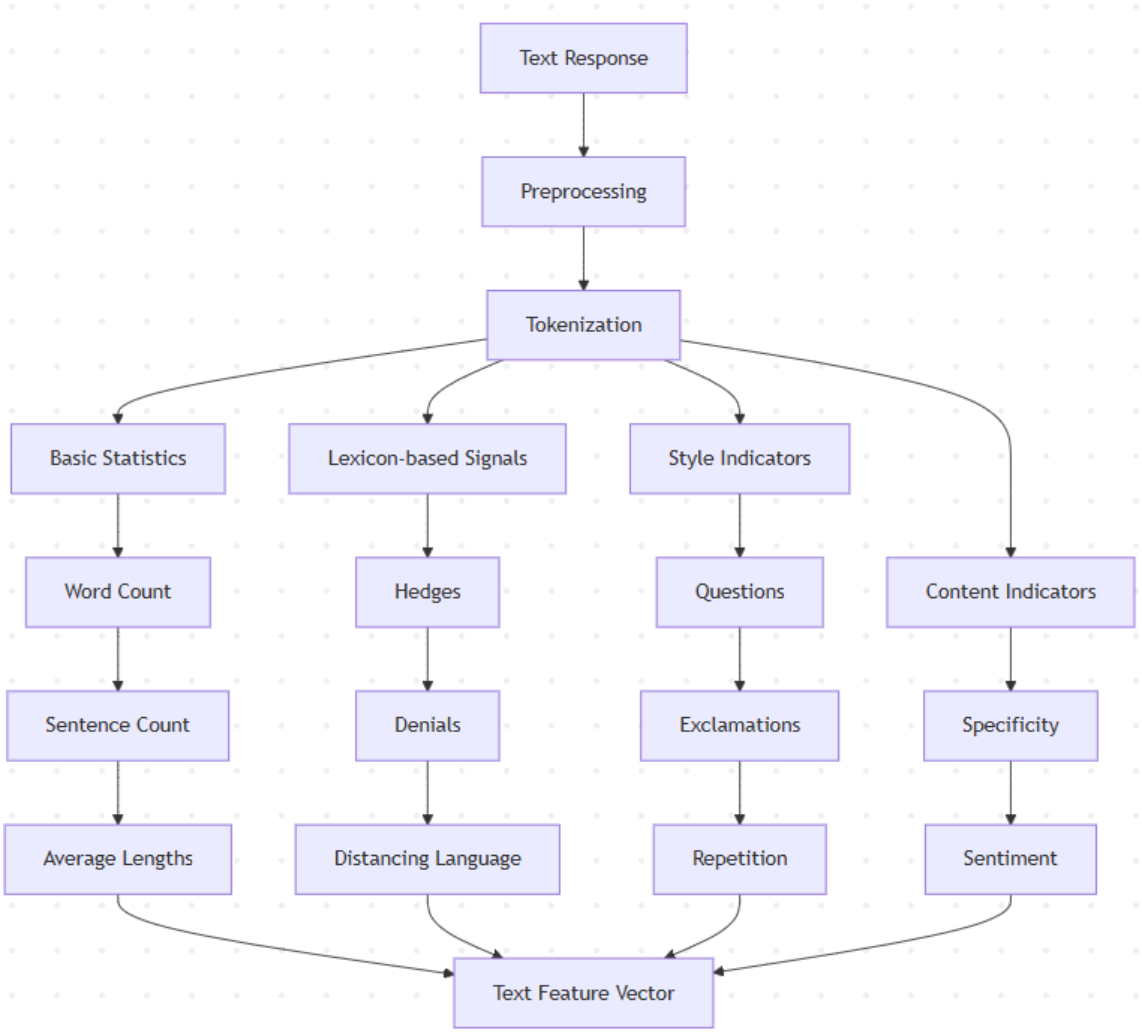
The inclusion of these features in the same feature vector is what makes the approach multi-modal. Not only is language considered, but also some physiological values are taken into account by the model.

Text Feature Extraction

Text Feature Extraction converts a raw response to linguistic features. It is critical at this stage because the success of modeling largely depends on extracted features; if they are not meaningful, models will not be able to learn from them.

The feature extraction method that was chosen for the project is rule-based and lexicon-based. The program detects various types of pre-defined features of speech including hedging words, denial words, distancing expression, and calculates structural features of the response, e.g., average sentence length, number of exclamation marks, redundancy of answers, etc.

One significant benefit of such a choice is that the features generated by this method are explainable, which is extremely important for fraud detection applications. In contrast to a neural network with millions of parameters, in this case, it will be possible to explain why a particular feature is considered suspicious and show a signal.



An effective code example will be the frontend of the feature extraction algorithm. It helps see how text can be turned into quantifiable features.

```
window.extractTextFeatures = function (text) {
  const tl = (text || "").toLowerCase();
  const words = tl.split(/\s+/).filter(w => w.length > 0);
  const sentences = (text || "").split(/[.!?]+/)
    .map(s => s.trim())
    .filter(s => s.length > 0);

  const nWords = words.length;
  const hedgeCount = count(L.HEDGES);
  const denialCount = count(L.DENIALS);
  const uniqueWords = new Set(words).size;

  return {
    n_words: nWords,
    n_sentences: Math.max(sentences.length, 1),
    hedge_ratio: nWords > 0 ? hedgeCount / nWords : 0,
    denial_ratio: nWords > 0 ? denialCount / nWords : 0,
    repetition_ratio: nWords ? 1 - uniqueWords / nWords : 0,
    avg_sentence_len: nWords / Math.max(sentences.length, 1)
  };
};
```

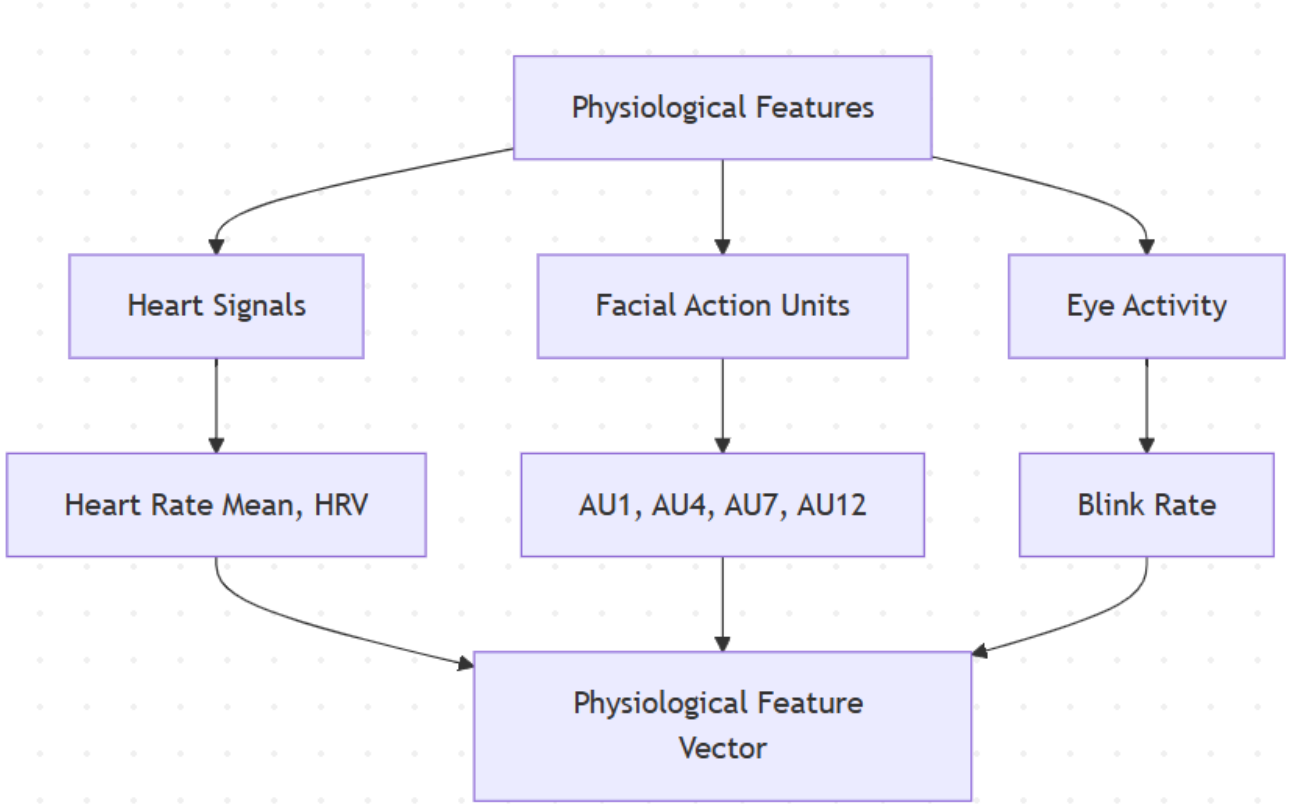
The reason why this part of the code is effective is that it provides a practical demonstration of how theory can be put to practice, showing how the model was created using engineered signals.

Physiological Features

These characteristics of the system are included because the system is supposed to be multimodal. In other words, while there might be some changes in the language used, some of them could be caused by physiological phenomena related to stress, high cognitive load, or effort to control emotions. In such a way, VERITY models these phenomena using heart rate, HRV, blink rate, and facial action units.

In the case of the machine learning implementation, the physiological information will be based on synthetic data. The advantage of using such data is that the system does not depend on having real physiological information available at the time of training. The information will include values that reflect what the system can expect from deceptive and truthful responses. For instance, deceptive behavior is characterized by higher heart rate, lower HRV, and more tense facial action units.

Nevertheless, the use of artificial physiological information is one of the biggest shortcomings of the proposed solution. Using such data, we can ensure the good performance of the system, but real physiological data is likely to be very noisy and affected by many external factors.



A simple piece of code may be written to demonstrate the simulation of physiological values when real signals are not available.

```
window.simulatePhysiological = function (truthBias = 0.5, t =
0) {
  const B = window.VERITY_BASELINES;
  const out = {};

  for (const [feat, b] of Object.entries(B)) {
    const base = b.truth + (b.decep - b.truth) * truthBias;
    const noise = (
      Math.sin(t * 0.001 + feat.length) * 0.5 +
      (Math.random() - 0.5)
    ) * b.noise;

    out[feat] = +(base + noise).toFixed(3);
  }

  return out;
};
```

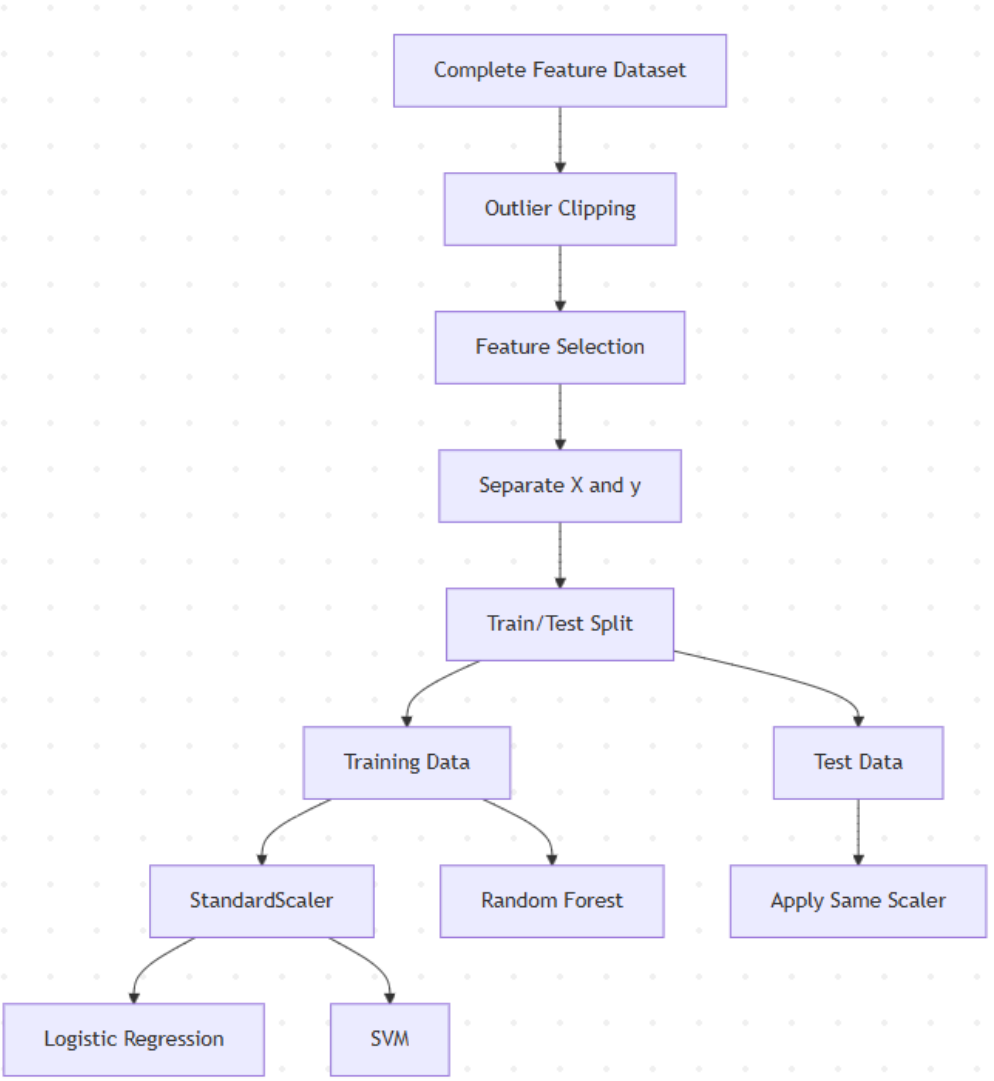
This piece of code is relevant since it proves why this model is possible even in the absence of real sensors. It also reinforces the limitation on the partial simulation of the physiological layer.

Data Preparation & Preprocessing

However, before training the models, data needs to be preprocessed. Outlier clipping using the interquartile range technique will be used in this project since the presence of outliers can make some model learnings incorrect. Since the dataset in this task is relatively small, this step is necessary.

Then, the data will be split into the training and testing datasets and divided according to the ratio between the number of features and labels. The stratify parameter will be set equal to y since stratification will ensure that the share of truthful and deceptive examples is the same in both datasets.

Feature scaling using StandardScaler is used since there are three algorithms: Logistic Regression, SVM, and Random Forest that need it. The only one that requires it is random forest since its operation is based on decision trees.



The code below represents the key preprocessing choices made.

```
for col in FEATURE_COLS:
   Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
   IQR = Q3 - Q1
   df[col] = df[col].clip(Q1 - 3 * IQR, Q3 + 3 * IQR)

X_train, X_test, y_train, y_test = train_test_split(
   X, y,
   test_size=0.25,
   stratify=y,
   random_state=42
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

This preprocessing step is significant in that it proves the project does not just train the models but actually prepares for training through a process of control.

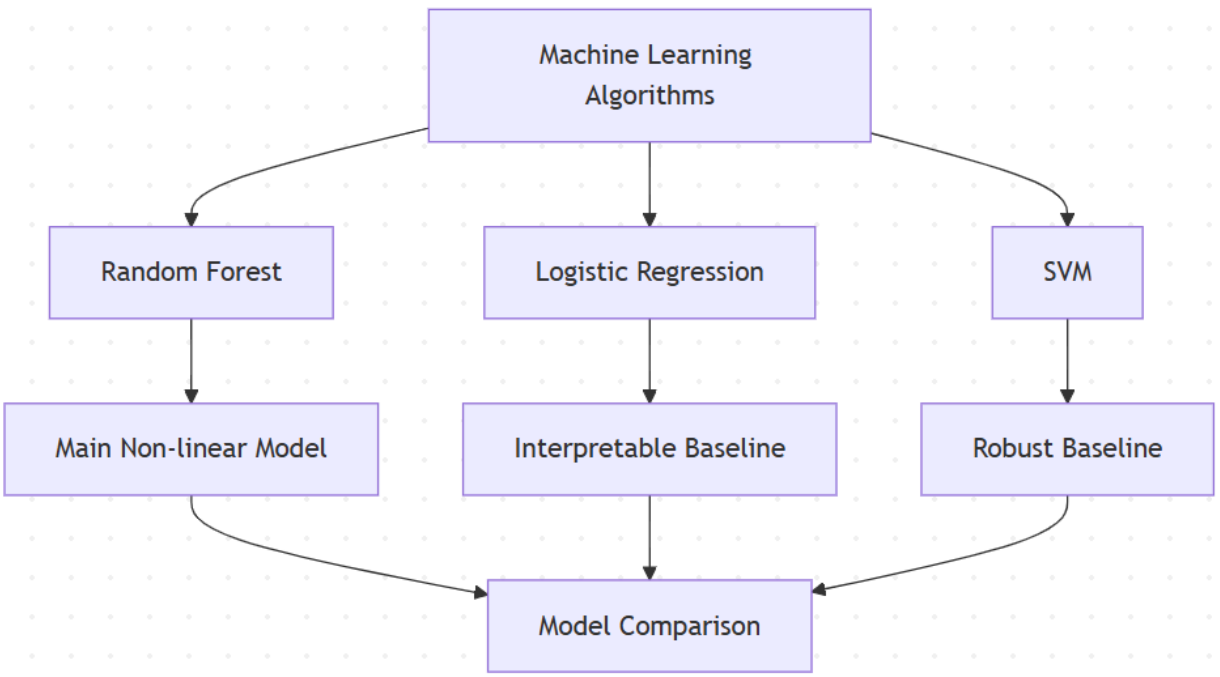
Machine Learning Algorithms

The VERITY experiment employs three types of supervised learning algorithms, Random Forest, Logistic Regression and Support Vector Machine. The role of each algorithm is distinct in this experiment.

Random Forest is the primary model for the task. It is an ensemble algorithm, based on multiple decision trees, and uses the results of individual models to come up with the final prediction. This method works well because it can deal with non-linear dependencies between variables. Furthermore, it doesn't need scaling the data and gives a measure of the feature importance.

Logistic regression is chosen as a simple classifier. It attempts to compute the probability of the instance being a lie using weighted averages of the input variables. Good performance of this model indicates that the classes are linearly separable.

Finally, Support Vector Machine acts as another baseline classifier for the project. It attempts to find a separation line, dividing the classes into two subsets of the examples. To facilitate getting probability estimates, the probability output in this model is switched on.



The model definitions must also be incorporated since they distinctly illustrate the algorithms along with their configurations.

```
rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    min_samples_leaf=2
)

lr = LogisticRegression(
    max_iter=500,
    random_state=42
)

svm = SVC(
    probability=True,
    random_state=42
)

rf.fit(X_train, y_train)
lr.fit(X_train_sc, y_train)
svm.fit(X_train_sc, y_train)
```

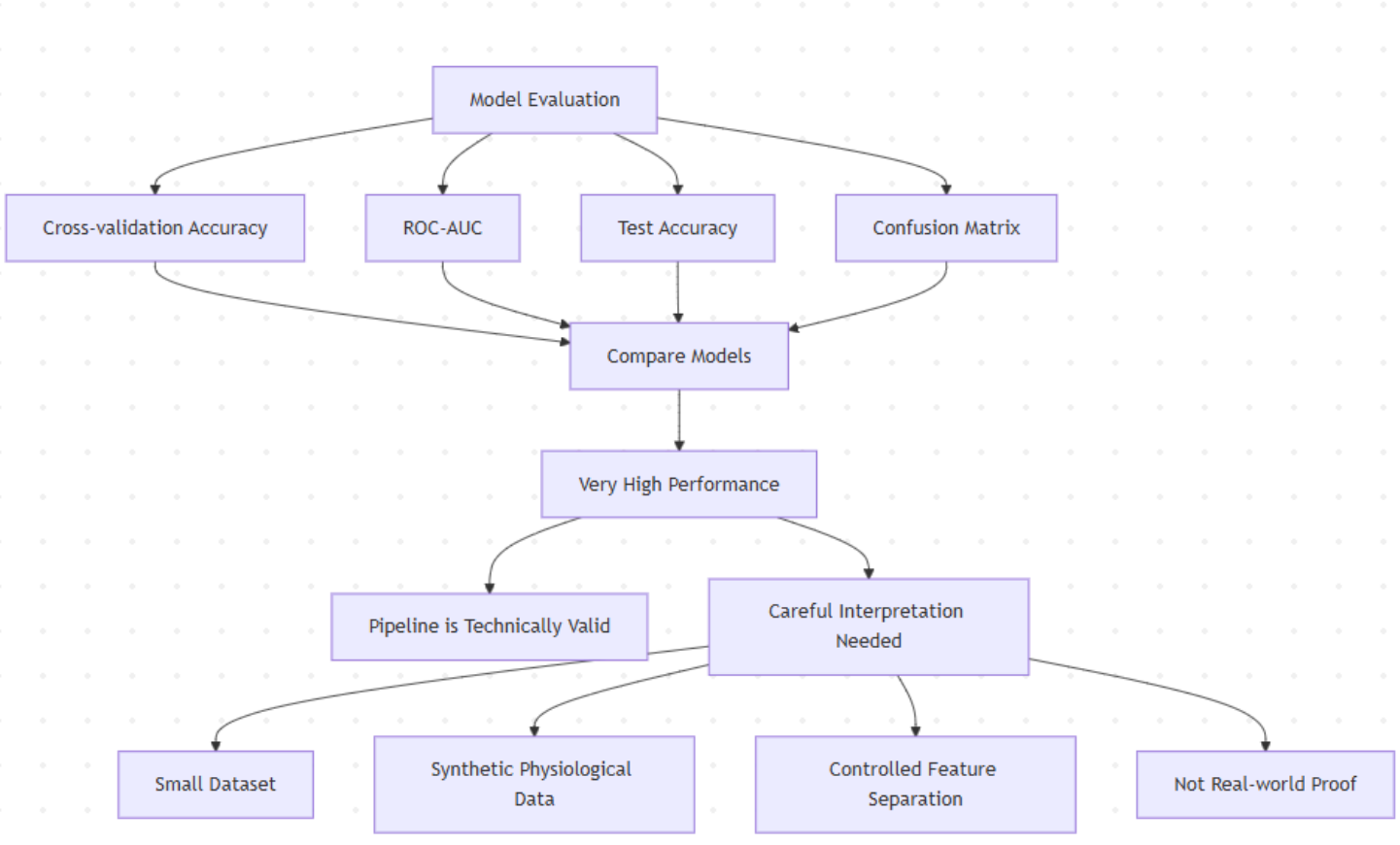
The following is another reason for the existence of two sets of training data: the original values are used by Random Forest, while logistic regression and SVM utilize the scaled values.

Model Evaluation & Interpretation

The models will be analyzed by means of cross-validation accuracy, ROC-AUC, test accuracy, and confusion matrices. Those evaluation techniques will provide some insight into how successfully the classifiers can distinguish between the two classes. For instance, accuracy evaluates the number of true predictions in relation to the overall number, whereas ROC-AUC reflects the classification capacity of the model regardless of threshold levels.

The project delivers impressive results. This indicates that the developed solution operates well and the engineered features have proven highly effective in the analysis of the data at hand. However, one must note that those results should not be regarded as the actual accuracy of the deception detection technique used in reality. The data set is relatively small and balanced, while physiological signals are artificial.

This part of the report should concentrate on proper interpretation of the results obtained.



The evaluation loop is a helpful piece of code to include, as it demonstrates that the project follows the same evaluation process for all models.

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)

for name, model, Xtr, Xte in [
    ("Random Forest", rf, X_train, X_test),
    ("Logistic Regression", lr, X_train_sc, X_test_sc),
    ("SVM", svm, X_train_sc, X_test_sc),
]:
    cv_acc = cross_val_score(model, Xtr, y_train, cv=cv,
scoring="accuracy")
    cv_auc = cross_val_score(model, Xtr, y_train, cv=cv,
scoring="roc_auc")

    yp = model.predict(Xte)
    ypr = model.predict_proba(Xte)[:, 1]
```

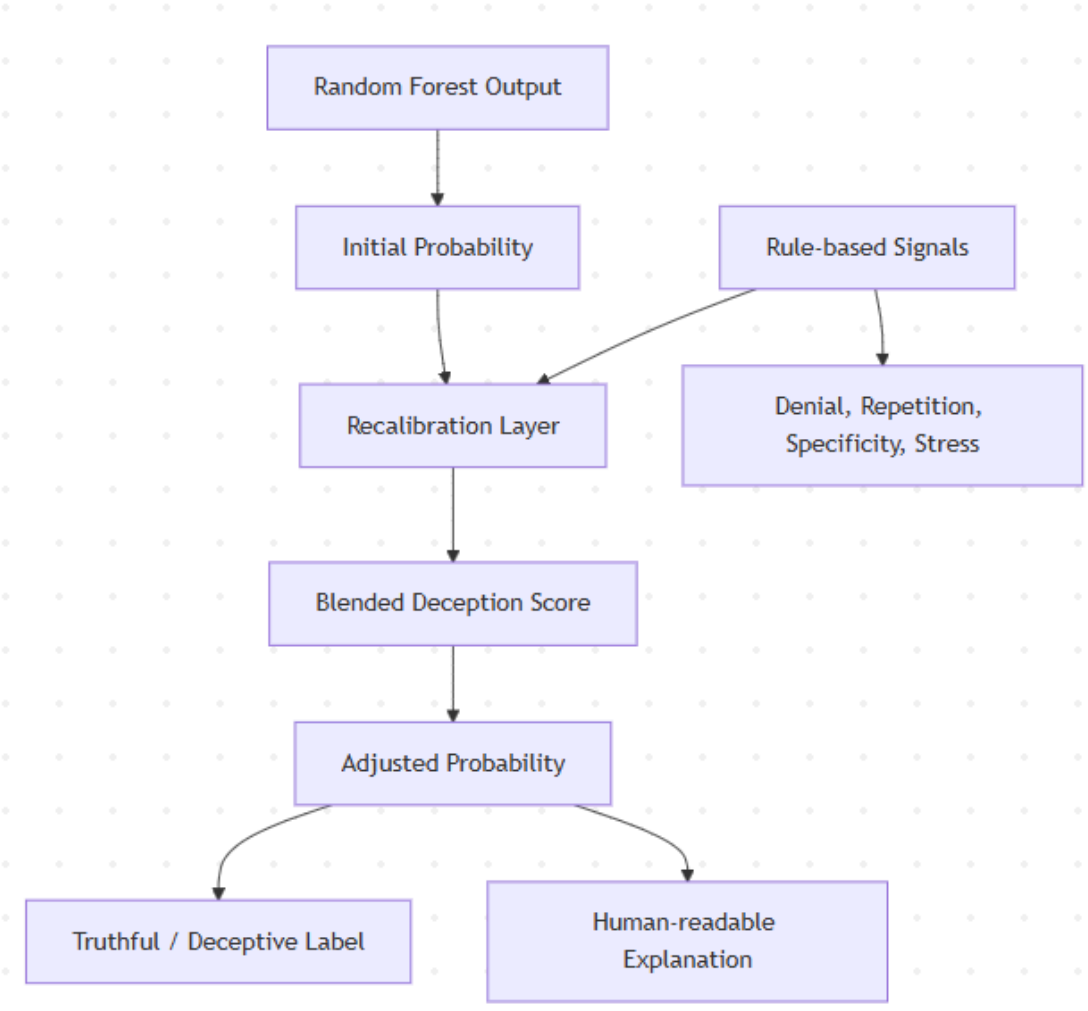
This portion of the report should also point out the advantage of the Random Forest algorithm, as it allows for calculating feature importance.

Recalibration Layer &Explainability

Following the Random Forest prediction, the VERITY algorithm adds a recalibration layer. This is one of the most exciting technological choices implemented in the research. The recalibration layer is based on combining the model's probability with rule-based reasoning. Thus, the final result will become more transparent and responsive to strong signals occurring during brief interrogation-style exchanges.

Rule-based reasoning tests if some specific features exceed certain thresholds. For example, an increased denial ratio, repetitions ratio, usage of many exclamation marks, lack of specificity, or presence of physiological stress markers increases the score assigned to deceitfulness. As a result, the prediction is a mixture of statistical algorithms and human reasoning.

A combination of the two approaches has pros and cons. The benefits lie in enhanced transparency and improved system performance in particular cases. Drawbacks arise from the possibility of introducing biases in manually developed rules.



This is a key functionality in terms of indicating how the hybrid approach was used in designing VERITY.

```
def recalibrated_score(feats):
    feat_vec = np.array([[feats[c] for c in FEATURE_COLS]])
    rf_prob = rf.predict_proba(feat_vec)[0]

    rule_score = 0.0
    total_weight = 0.0

    for feat, (direction, threshold, weight) in
THRESHOLDS.items():
        val = feats.get(feat, 0)

        if (direction == "high" and val > threshold) or \
           (direction == "low" and val < threshold):
            rule_score += weight

        total_weight += weight

    rule_decep = rule_score / total_weight if total_weight >
0 else 0.5
    blended_decep = 0.55 * rf_prob[1] + 0.45 * rule_decep

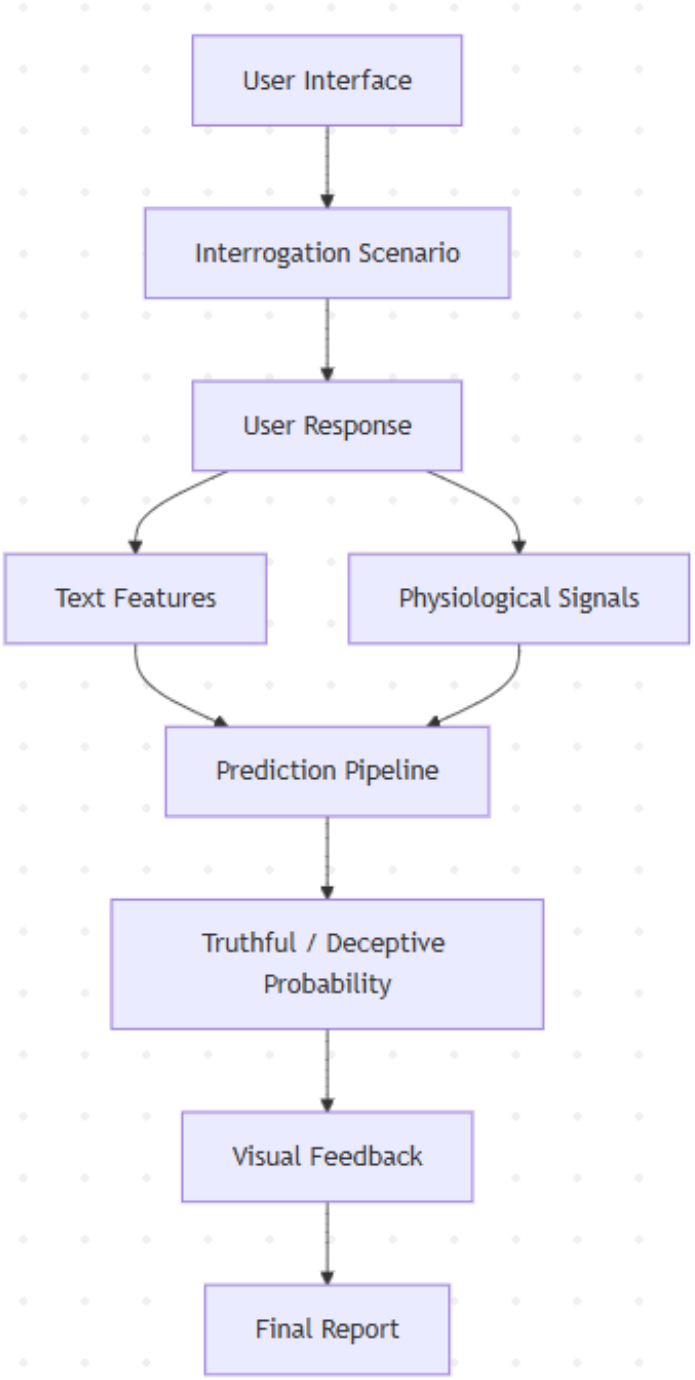
    blended_truth = 1.0 - blended_decep
    pred = 1 if blended_decep >= 0.5 else 0

    return pred, np.array([blended_truth, blended_decep]),
feats
```

In this case, explainability cannot be limited to being visual because it belongs to the model's logic, and therefore it can link certain attributes to the overall outcome prediction.

User Interface as Demonstration Layer

The user interface (UI) is not the project's key output but is essential because it represents the interactive potential of the proposed solution. In particular, VERITY has a cockpit-style user interface which includes the interrogation flow, the visualization of signal panels, probabilities, and the results. The other version of the application included a simpler Streamlit UI but the more advanced one was chosen in order to present the idea in full.



The UI can be described as a demonstration layer that bridges between the machine learning algorithm on one hand and an interactive user experience on another. The interface provides users with the ability to interact, receive questions, and view the simulated/generated signals as well as get the classification result. Some advanced capabilities like facial recognition, heart rate monitoring, speech analysis, and signal processing, although partially developed, were hard to stabilize completely, yet there is an underlying architecture for them.

It must be mentioned because in my view it makes the project much better than a mere notebook implementation. However, the UI must not become the focus of the project in terms of effort. Its purpose is to demonstrate the model and provide an intuitive way of understanding it.

This snippet of code will help demonstrate how the UI is integrated into the feature extraction and deception score calculation process.

```
const tf = window.extractTextFeatures(finalResponse || "I
don't know.");

let result = window.computeDeceptionScoreWithBaseline(
  tf,
  physForScoring,
  personalBaseline
);
```

As can be seen from the code, the UI plays an integral role, and it performs actions such as extracting text features, combining them with physiological data, and calculating a deception score.

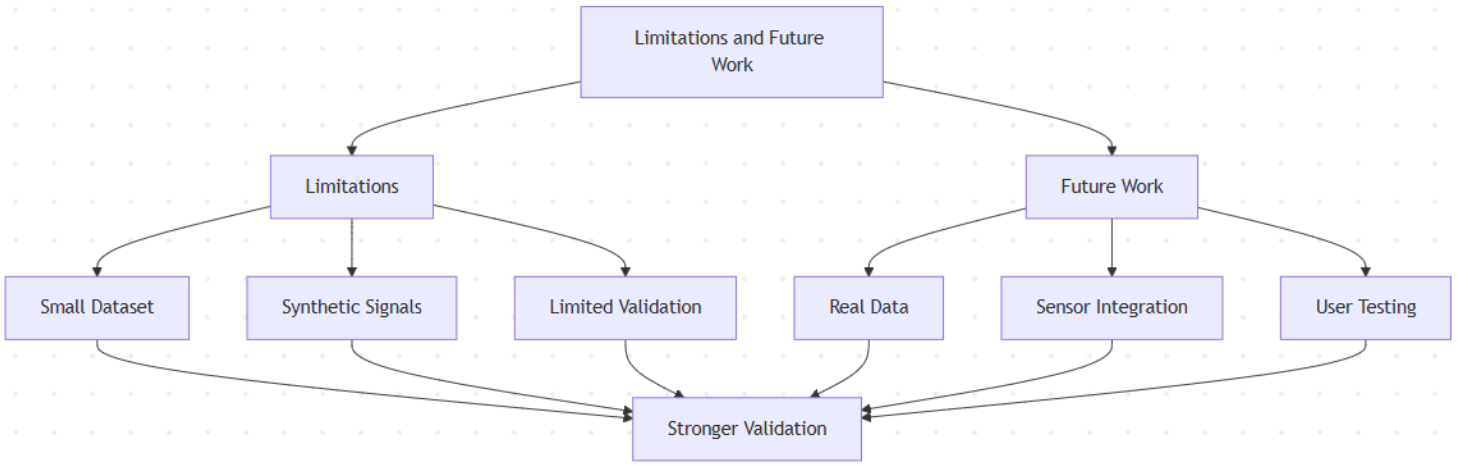
Limitations & Future Work

Several critical limitations have emerged during the creation of VERITY V1. First of all, the size of the dataset is not enough to prove any significant generalizations. Although it was sufficient to show the pipeline work, more variety in responses and more realistic data should be included in further projects.

Secondly, the project uses artificially generated physiological parameters which makes it simpler to test. However, this also means that the problem itself becomes simpler since the signals are clean without additional noise caused by real heart rate, HRV, blink rate, and facial signals influenced by different environmental circumstances.

Thirdly, although all advanced UI features are designed in the project, they still need more time to be fully developed. Namely, there were plans to integrate facial detection, voice analysis, heart-rate-related information, which will require some time to work properly.

All the limitations mentioned above should be taken into consideration in future work, especially regarding real data collection, model validation, comparison with other models using text, physiological, or combined methods, and evaluating the importance of the recalibration step.



The final section should be both realistic and positive. The research was not done for practical purposes, but it is an academic model which showcases a realistic machine learning process and highlights how it can be improved.

Conclusions

VERITY V1 provides insights into how a multimodal ML system is capable of predicting whether the response is true or false. This project presents an entire machine learning pipeline – from data collection and feature extraction to preprocessing, training of models, evaluating the results and recalibrating the classifier. The value of the project goes beyond the end result as it demonstrates the full process of creating a machine learning classification pipeline.

One of the best parts of this project is that the machine learning pipeline is used properly. The project trains, compares and evaluates performance of Random Forest, Logistic Regression and SVM based on the same set of features. Random Forest is chosen as the primary model for the following reasons: it captures nonlinear dependencies, works well with various numerical features, no scaling needed and provides feature importance. Logistic regression and SVM work as good baselines.

Another contribution worth mentioning is the feature engineering process itself. The VERITY framework extracts features such as denial ratio, hedge ratio, repetition, specificity, sentiment, heartrate, HRV, blink ratio and facial action units for each response. The recalibration step contributes to the interpretation of the results since it incorporates random forest prediction and rule-based evidence.

It also involves developing a user interface that allows showing the system in action. Nevertheless, the user interface is just a presentation layer here, whereas the real essence of the project is the machine learning pipeline. Certain elements that work in real time, including face recognition, audio analysis, and heartbeat measurement, are not yet fully stable, yet this can be done through proper coding in the future.

VERITY V1 should be treated as an academic machine learning prototype rather than a real deception detector. While the findings prove the feasibility of the system architecture, further development of the project would require bigger data sets, physiologically grounded results, and more stable real-time signal processing.